

Relatório de Auditoria de Segurança — Ballistic Pro

Data: 29/08/2026

Escopo auditado: API FastAPI (api/), núcleo compartilhado (core/, services/, schemas.py), app Streamlit (app.py) e frontend React/PWA (web/), além dos arquivos de deploy (Dockerfile*, docker-compose.yml) e do pipeline de CI (.github/workflows/ci.yml).

Nota metodológica — stack detectada e mapeamento das categorias:

Backend Python **FastAPI** + **SQLAlchemy** (ORM), auth por **JWT** (PyJWT/HS256) sobre **bcrypt**, PII cifrada com **Fernet** e blind index HMAC. Frontend **React 18** + **Vite** + **TypeScript** (PWA). Deploy **Docker/EasyPanel**; CI em GitHub Actions. Não há Supabase/RLS.

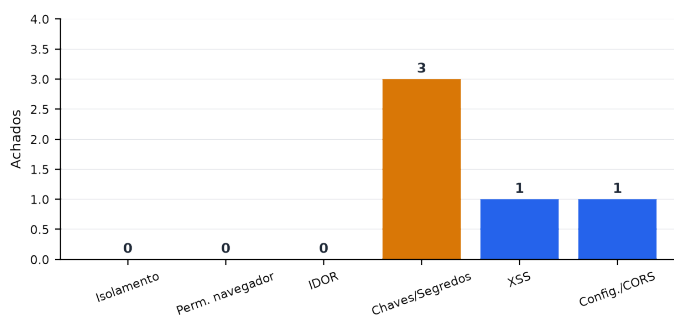
1) **Banco sem tranca** → isolamento é filtro manual por **user_id** (+ helper **_owned_or_404**); verificou-se sua presença em toda listagem/agregação/exportação. 2) **Permissão no navegador** → cruzou-se todo gate de UI com o endpoint; não há operação por papel (is_premium é só rótulo). 3) **IDOR** → percorreu-se todo handler que recebe ID (path/body). 4) **Chaves expostas** → código, configs, docker-compose, CI, templates e histórico git. 5) **XSS** → href/innerHTML/eval no frontend e HTML/e-mail no backend.

Resumo executivo

A auditoria cobriu as 5 categorias solicitadas, arquivo por arquivo. Foram confirmados **5 achados**: 0 crítica(s), 1 alta(s), 2 média(s) e 2 baixa(s). As categorias de **isolamento de dono**, **permissão no servidor** e **IDOR** não geraram achados — o backend aplica filtro por `user_id` e o padrão `_owned_or_404` de forma consistente e testada. Os riscos concentram-se em **gestão de segredos/defaults** (assinatura do JWT) e em **hardening** (URLs em href e CORS).



Alta (1) Baixa (2)
Média (2)



Esquerda: achados por severidade. Direita: achados por categoria auditada.

Pontos fortes (o que está protegido)

✓ Isolamento por dono em todos os routers de dados

data.py, activities.py, documents.py, events.py, places.py, dope.py, backup.py, insights.py e reports.py filtram toda listagem/agregação por `filter_by(user_id=current['id'])`. Não há RLS (não é Supabase); o mecanismo é filtro manual por `user_id + helper _owned_or_404`.

✓ IDOR fechado no padrão `_owned_or_404`

Toda rota que busca/edita/apaga objeto por ID (PUT/DELETE de firearms, documents, events, places, dope-cards; download de documento e etiqueta de logbook) verifica posse e responde 404 para objeto alheio. Referências cruzadas (firearm_id em activities/dope/logbook) revalidam a posse da arma.

✓ Cobertura de testes de isolamento

Há testes `test_*_isolation / firearm_must_belong_to_user` em `test_api_data, _activities, _documents, _events, _places, _dope, _backup, _reports` — cross-user PUT/DELETE retorna 404.

✓ Sem superfície de privilégio no cliente

Não há operação administrativa/por papel no app. `is_premium` é apenas rótulo de exibição (Profile.tsx). Logo, não existe gate de permissão só-no-navegador a ser burlado (categoria 2 = N/A).

✓ Recuperação de senha robusta

PasswordReset guarda só o hash SHA-256 do token, com expiração de 1h e uso único; resposta sempre genérica (anti-enumeração). Senhas via bcrypt; logout de força-bruta persistido no servidor.

✓ PII cifrada em repouso, com trava de produção

EncryptedString (Fernet) cifra serial/CRAF/GTS/CPF/e-mail; `get_encryption_suite RECUSA` iniciar em produção sem chave. Campos pesquisáveis usam blind index HMAC.

✓ WebAuthn correto

Servidor guarda apenas a chave pública; desafio de uso único com TTL de 10 min; `sign_count` verificado; `login/complete` confere `credential_id + user_id`.

✓ Sem segredos versionados e sem chave no bundle

`.env.example` e `secrets.toml.template` só têm placeholders; nenhuma FERNET/JWT/AWS/Resend real no histórico git; o bundle do frontend não embute chaves (usa `VITE_API_URL` para a base).

✓ Sem vetores clássicos de XSS

Nenhum `dangerouslySetInnerHTML/innerHTML/eval` no frontend; React escapa por padrão. E-mails são enviados como texto puro (`services/mailer.py`), sem HTML com input do usuário.

Pontos fracos (riscos centrais)

O eixo de maior risco é a **assinatura do JWT**: há um segredo de desenvolvimento embutido como fallback silencioso (F1) e, no caminho de produção documentado, a própria chave de criptografia é reutilizada para assinar tokens (F2). Em segundo plano, há **defaults de segredo/credencial** (F3) que só não viram brecha porque dependem de sobrescrita. Por fim, itens de **hardening**: URLs de usuário em href sem allowlist (F4, hoje self-XSS) e CORS `**` padrão (F5).

Achados detalhados

Sev.	Arquivo:linha	Descrição
Alta	F1 api/security.py:35, 38–47	Segredo do JWT cai para um valor de desenvolvimento embutido, sem rejeição no startup
Média	F2 api/security.py:39	Chave de criptografia de PII (FERNET_KEY) reutilizada como segredo de assinatura do JWT
Média	F3 core/models.py:38, 720; docker-compose.yml	Segredos e credenciais padrão embutidos (blind index dev, admin padrão, DB postgres:postgres)
Baixa	F4 web/src/pages/Places.tsx:132; Events.tsx:112; Documents.tsx:209; Acervo.tsx:164,170	URLs controladas pelo usuário em href sem allowlist de protocolo (self-XSS)
Baixa	F5 api/main.py:43–50	CORS liberado com '*' por padrão

F1 — Segredo do JWT cai para um valor de desenvolvimento embutido, sem rejeição no startup

Alta Categoria: Chaves/Segredos · Local: api/security.py:35, 38–47

Por que é explorável: Se JWT_SECRET e FERNET_KEY estiverem ambos ausentes no ambiente do serviço da API, os tokens passam a ser assinados com uma constante pública presente no código-fonte. Qualquer pessoa que leia o repositório pode forjar um JWT com sub= e ser autenticado como ele. Só há um warning — não há validação de startup que recuse o segredo padrão. O gatilho é plausível: a API lê apenas variáveis de ambiente; um deploy que configure a criptografia via .streamlit/secrets.toml (device_encryption_key), como sugere o próprio template do projeto, deixa a API sem FERNET_KEY/JWT_SECRET no ambiente e ativa o fallback silenciosamente.

Impacto: Falsificação de token e tomada de conta de qualquer usuário (bypass total de autenticação).

Evidência:

```
_DEV_SECRET = "dev-insecure-jwt-secret-change-me"

def _secret() -> str:
    secret = os.getenv("JWT_SECRET") or os.getenv("FERNET_KEY")
    if not secret:
        warnings.warn("[SECURITY] ... usando segredo de desenvolvimento ...")
        return _DEV_SECRET
    return secret
```

Correção sugerida: Exigir JWT_SECRET explícito. No startup, recusar iniciar (raise) se o segredo resolvido for _DEV_SECRET ou vazio quando o ambiente for de produção (mesma heurística já usada em get_encryption_suite). Não derivar o segredo do JWT da chave de criptografia (ver F2).

F2 — Chave de criptografia de PII (FERNET_KEY) reutilizada como segredo de assinatura do JWT

Média Categoria: Chaves/Segredos · Local: api/security.py:39

Por que é explorável: No caminho de produção documentado, define-se FERNET_KEY e não JWT_SECRET; então o mesmo material de chave que cifra os dados sensíveis (serial, CRAF, GTS, CPF) também assina os tokens de sessão. Isso viola separação de chaves: vazamento ou rotação de uma quebra a outra, e amplia o raio de dano de qualquer exposição da FERNET_KEY.

Impacto: Acoplamento de segredos: comprometer/rotacionar a chave de cifra afeta também toda a autenticação.

Evidência:

```
secret = os.getenv("JWT_SECRET") or os.getenv("FERNET_KEY")
```

Correção sugerida: Definir sempre JWT_SECRET próprio (independente da FERNET_KEY) e remover o fallback para FERNET_KEY, ou derivar subchaves distintas por HKDF a partir de um segredo mestre.

F3 — Segredos e credenciais padrão embutidos (blind index dev, admin padrão, DB postgres:postgres)

Média

Categoria: Chaves/Segredos · **Local:** core/models.py:38, 720; docker-compose.yml

Por que é explorável: São defaults do tipo \${VAR:-valor} ou constantes de código que viram segredo real se não forem sobrescritos. A criação do admin padrão e a cifra têm trava de produção (is_production), mas a chave do blind index e as credenciais do Postgres não são recusadas no startup. O admin é criado pelo app Streamlit (app.py) e, como API e Streamlit compartilham o mesmo banco, o usuário 'atirador_pro' com senha conhecida fica logável também pela API se o app subir em modo não-produção contra um banco alcançável.

Impacto: Login com credencial padrão conhecida; se a porta do Postgres for exposta, acesso direto ao banco com postgres:postgres; blind index previsível enfraquece a unicidade/pesquisa cifrada.

Evidência:

```
# core/models.py:38 (chave do blind index)
raw = "ballistic-pro-dev-blind-index"

# core/models.py:720 (senha do admin criado no 1º boot)
admin_pass = "ballistic_admin_2025!" # user "atirador_pro"

# docker-compose.yml (defaults ${VAR:-...})
DATABASE_URL=...postgres:postgres@db:5432/...
POSTGRES_PASSWORD=${POSTGRES_PASSWORD:-postgres}
```

Correção sugerida: Recusar startup em produção quando o blind index/DB usarem os defaults; remover a senha de admin embutida (exigir ADMIN_PASSWORD sempre); documentar POSTGRES_PASSWORD forte e nunca expor a porta 5432 publicamente.

F4 — URLs controladas pelo usuário em href sem allowlist de protocolo (self-XSS)

Baixa

Categoria: XSS · **Local:** web/src/pages/Places.tsx:132; Events.tsx:112; Documents.tsx:209; Acervo.tsx:164,170

Por que é explorável: O React escapa texto, mas não sanitiza o protocolo de href. O usuário pode salvar 'javascript:...' nos campos de link (site do local, URL do evento, link do documento/CRAF/GTS). Ao clicar, o script executa na origem do app. Hoje esses registros são exibidos apenas na sessão do próprio dono (dados de um único inquilino), então é self-XSS.

Impacto: Execução de script na origem do app ao clicar no link. Baixa por ser self-XSS; sobe para stored-XSS contra terceiros se os registros passarem a ser compartilhados (ex.: acervo de clube) ou exibidos em telas administrativas/relatórios de outro usuário.

Evidência:

```
// Places.tsx
<a href={p.url} target="_blank" rel="noreferrer">site</a>
// Acervo.tsx
<a href={f.craf_doc_url} ...>CRAF</a> <a href={f.gts_doc_url} ...>GTS</a>
// Documents.tsx: href={d.file_url} // Events.tsx: href={ev.url}
```

Correção sugerida: Criar um helper safeHref(url) que só aceita http/https/tel/mailto e devolve undefined caso contrário; aplicar em todos os href de dado do usuário. Opcional: validar o protocolo também no backend ao salvar.

F5 — CORS liberado com '*' por padrão

Baixa

Categoria: Configuração/CORS · **Local:** api/main.py:43–50

Por que é explorável: O default abre a API para qualquer origem. O risco é limitado porque a autenticação é por Bearer token (não cookie) e allow_credentials fica desligado quando origem é '*', então um site malicioso não lê o

token da vítima. Ainda assim, expõe a superfície da API a chamadas de qualquer origem e é uma configuração insegura por padrão.

Impacto: Superfície de API acessível de qualquer origem no navegador; hardening pendente.

Evidência:

```
_origins = os.getenv("API_CORS_ORIGINS", "")
allow_origins = ["" if _origins.strip() == "" else [...]]
app.add_middleware(CORSMiddleware, allow_origins=allow_origins,
    allow_credentials=_origins.strip() != "", allow_methods=["*"], ...)
```

Correção sugerida: Definir API_CORS_ORIGINS com a lista de origens do frontend em produção e recusar '*' quando em produção.

Recomendações priorizadas

Prioridade	Sev.	Ação
P1	Alta	Tornar JWT_SECRET obrigatório e recusar startup com o segredo de desenvolvimento (F1). Parar de reutilizar FERNET_KEY para assinar JWT (F2).
P2	Média	Eliminar defaults de segredo/credencial (F3): remover senha de admin embutida, recusar blind index/DB padrão em produção, senha forte de Postgres e porta não exposta.
P3	Baixa	Sanear URLs de usuário em href com allowlist de protocolo (F4) e restringir CORS a origens confiáveis em produção (F5).

Issues para o GitHub

Cada bloco abaixo é o texto completo de uma issue, pronto para copiar e colar. Achados de segredos afins (F1–F3) foram agrupados numa issue única para não gerar spam.

```
--- ISSUE 1 ---
# [Segurança] Endurecer a gestão de segredos: JWT, chave de cifra e defaults

**Labels:** security, alta

## Problema
Três pontos ligados a segredos/credenciais:
- **F1 (api/security.py:35,38-47):** o segredo do JWT cai para a constante embutida `dev-insecure-jwt-secret-change-me` quando `JWT_SECRET` e `FERNET_KEY` estão ausentes no ambiente da API – apenas com um warning, sem recusar o startup. A API lê só variáveis de ambiente; configurar a cifra via .streamlit/secrets.toml (como sugere o template) deixa a API sem as chaves e ativa o fallback. Com o segredo público, qualquer um forja um JWT (sub = id de qualquer usuário) e assume a conta.
- **F2 (api/security.py:39):** `secret = getenv("JWT_SECRET") or getenv("FERNET_KEY")` reutiliza a chave de cifra de PII para assinar o JWT (sem separação de chaves).
- **F3 (core/models.py:38,720; docker-compose.yml):** defaults embutidos – blind index `ballistic-pro-dev-blind-index`, senha do admin `ballistic_admin_2025!` (user `atirador_pro`) e Postgres `postgres:postgres`.

## Evidencia
- api/security.py:35   `_DEV_SECRET = "dev-insecure-jwt-secret-change-me"`
- api/security.py:39   `secret = os.getenv("JWT_SECRET") or os.getenv("FERNET_KEY")`
- core/models.py:720   `admin_pass = "ballistic_admin_2025!"`
- docker-compose.yml   `POSTGRES_PASSWORD=${POSTGRES_PASSWORD:-postgres}`

## Impacto
Falsificação de token / tomada de conta (F1); acoplamento de segredos (F2); login com credencial padrão e acesso ao banco com senha default (F3).

## Correção sugerida
- Exigir `JWT_SECRET` próprio e distinto; no startup, **recusar iniciar** se o segredo resolvido for o default ou vazio em produção (reusar a heurística is_production de get_encryption_suite). Não derivar o JWT da FERNET_KEY.
- Remover a senha de admin embutida (exigir ADMIN_PASSWORD sempre) e recusar os defaults de blind index/Postgres em produção. Nunca expor a porta 5432.

## Critérios de aceite
- [ ] API não inicia em produção sem JWT_SECRET (falha explícita, não warning).
- [ ] `_DEV_SECRET` nunca é usado quando há DATABASE_URL postgres ou FERNET_KEY.
- [ ] JWT_SECRET e FERNET_KEY são valores distintos; o código não assina JWT com FERNET_KEY.
- [ ] Admin padrão não é criado sem ADMIN_PASSWORD; sem credenciais default utilizáveis no compose.
- [ ] Teste cobre a recusa de startup com segredo ausente/padrão.
--- FIM ISSUE 1 ---
```



```
--- ISSUE 2 ---
# [Seguranca] Sanear URLs de usuario em href (allowlist de protocolo)

**Labels:** security, baixa

## Problema
Campos de link controlados pelo usuario sao renderizados direto em `href`, sem
validar o protocolo. Um valor `javascript:...` executa script na origem do app
ao ser clicado. Hoje os registros sao vistos so pelo proprio dono (self-XSS),
mas vira stored-XSS contra terceiros se houver compartilhamento (ex.: acervo de
clube) ou telas administrativas.

## Evidencia
- web/src/pages/Places.tsx:132 `<a href={p.url}>`
- web/src/pages/Events.tsx:112 `<a href={ev.url}>`
- web/src/pages/Documents.tsx:209 `<a href={d.file_url}>`
- web/src/pages/Acervo.tsx:164,170 `<a href={f.craf_doc_url}>` / `{f.gts_doc_url}`

## Impacto
Execucao de script na origem do app ao clicar; escalada a XSS armazenado se os
dados passarem a ser exibidos para outros usuarios.

## Correcao sugerida
Criar `safeHref(url)` que so aceita `http/https/tel/mailto` (devolve undefined
caso contrario) e aplicar em todos os href de dado do usuario. Opcional: validar
o protocolo tambem no backend ao salvar.

## Criterios de aceite
- [ ] Helper de saneamento aplicado em todos os href de dado do usuario.
- [ ] URLs `javascript:``/data:` nao viram href navegavel.
- [ ] Teste cobre que `javascript:...` nao gera href ativo.
--- FIM ISSUE 2 ---
```

```
--- ISSUE 3 ---
# [Seguranca] Restringir CORS a origens confiaveis em producao

**Labels:** security, baixa

## Problema
`API_CORS_ORIGINS` tem default `` (api/main.py:43-50), abrindo a API para
qualquer origem. O risco e limitado porque a auth e por Bearer token (nao
cookie) e allow_credentials fica off com ``, mas e uma config insegura por
padrao.

## Evidencia
- api/main.py:43 `_origins = os.getenv("API_CORS_ORIGINS", "")`
- api/main.py:49 `allow_credentials=_origins.strip() != ""`

## Impacto
Superficie de API acessivel de qualquer origem no navegador; hardening pendente.

## Correcao sugerida
Definir `API_CORS_ORIGINS` com as origens do frontend em producao e recusar ``
(ou logar como erro) quando o ambiente for de producao.

## Criterios de aceite
- [ ] Em producao, apenas origens confiaveis sao aceitas.
- [ ] O default `` e recusado/alertado quando o ambiente e de producao.
--- FIM ISSUE 3 ---
```